SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

Chapter 4 — Data Description and Visualization
Descriptive statistics, chart choice, dashboards and honest graphs

Lecturer: Xieer Dai (David)

Shenzhen University • Weeks 3–14 · Thursday, periods 11–12



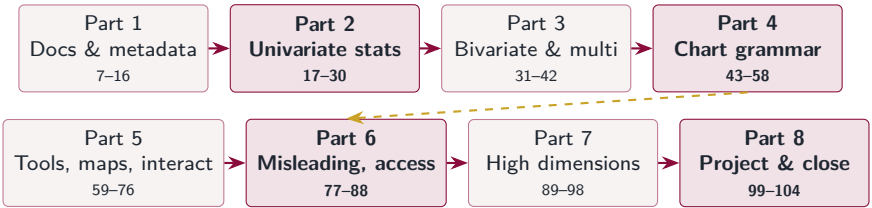
Learning Objectives

After this lecture you should be able to:

1. write the data documentation and metadata at the very start of a project (including the 20-item checklist and a field dictionary);
2. describe a variable completely with the trio of central tendency, dispersion and distribution shape, and state how robust each of them is;
3. use correlation, contingency tables and conditional distributions correctly in bivariate and multivariate settings, and recognise the effect of a restricted range;
4. choose a chart from the grammar of graphics (data $\rightarrow$ geometry $\rightarrow$ channel) and a decision table, and write self-contained captions;
5. produce interactive charts, dashboards and geo-visualisations in Python (choropleth, symbol map, spatial autocorrelation);
6. recognise and fix misleading charts, build colour-blind friendly, accessible graphics, and report conclusions with a narrative structure.

Source: This lecture follows five threads: documentation and metadata, descriptive statistics, descriptive graphics, visualisation tools, maps and networks – extended to 2026 practice.

Roadmap (104 frames)



Spine: documentation → statistics → chart types → tools and maps → honest reporting → high-dimensional frontier

12 code frames · 18 sample frames: every page pairs sample data, a chart comparison and a line-by-line commented Python program – all in Python

From Preparation to Description

What you hold at the end of L3

- a clean analysis table (raw and curated layers);
- a cleaning log, missing-value records and reproducible scripts;
- but one question is still open: what does this data actually look like, and how do I explain it to others?

What L4 answers

- whose data is it, on what definition? How is documentation and metadata written?
- which statistics describe the distribution?
- which chart expresses this question?
- how do I draw it so that it does not mislead and everyone understands it?



Every program in this lecture is written in Python: no Power BI or Tableau. Interactive charts and dashboards are exported to HTML with plotly; maps use choropleth and symbol maps.

Deliverables

1) Data documentation

`README` + field dictionary (codebook): types, units, value domains, missingness, versions and checksums (code page 14)

2) Descriptive table

the central / dispersion / shape trio + grouped comparison tables; missing rates are reported with the table (code page 25)

3) Chart list and captions

for every chart state: question, data, chart type, channels, sample size, definition and uncertainty (template, slide 98)

Matching Project 1 of this course, "Data sourcing and preparation"

The report part of Project 1 uses exactly these three: documentation + descriptive table + chart list. The "ten checks for an honest chart" (slide 97) and the "AI-generated chart verification checklist" (slide 86) serve directly as your report self-check.

Source: Marking and submission requirements are in section 6 of the syllabus; this lecture supplies the templates and checklists.

Map of the Lecture

Part	Frames	Key question
Documentation and metadata	7–16	Whose data is it, on what definition? How is a field dictionary generated automatically?
Univariate descriptive statistics	17–30	Which measure for centre, dispersion and shape? Which is most robust?
Bivariate and multivariate	31–42	How do I read a correlation? Does grouping change the conclusion?
Chart choice and the grammar of graphics	43–56	Which chart for this question? How are channels chosen?
Tools, interactivity and geo-visualisation	57–74	How do I build interactive charts, dashboards and maps in Python?
Misleading charts, accessibility and narrative	75–86	How do charts lie? How do I make everyone understand them?
High-dimensional visualisation	87–96	How do I draw a dozen dimensions? Projections, parallel coordinates and interpretable graphics.
Project and wrap-up	97–102	What standard must the report's charts meet?

How to read it: every part is the cycle "concept → sample and figure (sample page) → runnable program (code page) → checklist".

Four Settings

Setting	Typical example	What the charts must do
Investment research report	Investment research report	Conclusion first, key metrics shown as a dashboard; the definition and the point in time must appear on the chart
Thesis writing	Thesis writing	Self-contained, reproducible captions; statistics and figures numbered separately, one style throughout the book
Scientific academic research	Scientific academic research	Follow the journal's figure rules; error bars, sample size and significance labels are all mandatory
Consulting firm report	Consulting firm report	One conclusion per page, a clear narrative; avoid chart junk and misleading axes

What they share: readers will not read your code, only your charts and captions. "Self-contained captions" is therefore one of the core requirements of this lecture (slide 51).

Someone will read the chart without you.

Source: The setting list is compiled from international-business practice; the chart requirements follow international journal and consulting-report conventions.

Documentation from Day One

Core principle

Start recording the data at the very beginning of a study – even before collection starts. That makes documentation easier and lowers the chance that you forget the details of the data later. At a minimum keep this documentation in a `readme.txt` or equivalent file stored with the data; you may also cite a published paper as a supplement.

The cost of skipping documentation

- three months later you cannot tell the unit of a field either;
- you cannot answer a reviewer's "what is the definition?";
- handing the work to a classmate means explaining everything by word of mouth;
- the data is misused and over-interpreted.

The link to L3's data contract: the contract is "machine-readable documentation" (types, value domains, missing-value conventions) and the documentation is the "human-readable contract" (background, method, licence, version) – the two should share one source and check each other.

Source: Documentation-first is standard data-management practice; the costs and the link to L3 are compiled for this lecture.

The 20-Item Checklist, Part 1

Item	Key description
Title	the name of the dataset or research project
Creator	name and address of the institution or people who created the data
Identifier	the identifier used to refer to the data, even an internal project number
Subject	keywords or phrases describing the subject or the content of the data
Funders	the institution or department that funded the research
Rights	the known intellectual-property rights in the data
Access information	where and how other researchers can obtain the data
Language	the language of the intellectual content (where applicable)
Dates	project start and end, release date, period covered by the data, maintenance and update cycle
Location	spatial coverage information for the physical locations the data refers to

Source: The 20-item data-documentation checklist (split over two frames for legibility, with a fill-in template).

The 20-Item Checklist, Part 2

Item	Key description
Methodology	how the data was produced: equipment or software, experimental procedure, notes comparable to a lab notebook
Data processing	any information about how the data was changed or processed, recorded as it happens
Sources	citations of the original sources of derived data, including where the source data is kept and how to get it
List of file names	names and extensions of all the project's data files (e.g. `stone.mov`)
File formats	the data formats (FITS, SPSS, HTML, JPEG, ...) and the software needed to read them
File structure	how the data files are organised and how the variables are laid out (where applicable)
Variable list	the list of variables in the data files (where applicable)
Code lists	explanation of the codes and abbreviations in file names or variables (e.g. `999 means missing`)
Versions	the timestamp of every file, with a separate ID for each version
Checksums	used to detect whether a file has been changed over time (use together with backups)

Source: As above: the second half of the checklist (Methodology to Checksums).

Metadata as Data about Data

Definition

"Metadata" is data about data: a structured description used to describe content and to make it easier to discover and use. Metadata records can be embedded in the data or kept separately; a data file in any format can carry metadata fields. In the social sciences this record is called a codebook or data dictionary.

A minimum viable data dictionary

- field name and its meaning;
- type and unit (CNY / thousand CNY / %);
- value domain (enumeration, range, coded values);
- missing conventions (-1, NULL, empty string);
- non-null rate and number of unique values.

The code pages of this lecture generate this dictionary automatically: change the data structure and the dictionary follows, so documentation and data never drift apart.

Source: The metadata definition follows standard data-management wording; the minimum viable dictionary fields correspond to code page l4_01.

Metadata Standards

Six common standards

- Astronomy Visualization Metadata (astronomy)
- Content Standard for Digital Geospatial Metadata (geospatial)
- Darwin Core (biodiversity)
- Data Documentation Initiative (social sciences)
- Dublin Core (general)
- Ecological Metadata Language (ecology)

Which one?

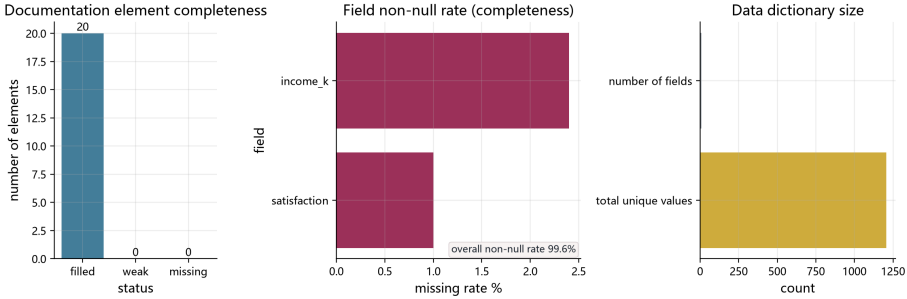
It depends on the type, scale and discipline of the research; the UK Digital Curation Centre (DCC) maintains discipline-by-discipline lists.

What if there is no standard

Build your own minimum metadata set from the 20-item checklist on slides 9–10 and fix it inside the team – consistency matters more than "being standard".

Source: The standard list and the DCC pointer follow each standard's official description; "what to do without a standard" is compiled for this lecture.

[Sample + Visualisation] Documentation Coverage



Left: coverage of the 20 documentation items (this lecture's sample is 20/20 complete); middle: the non-null rate of each field (income is a sensitive question – 12 missing, satisfaction 5 missing); right: the size of the auto-generated data dictionary (9 fields).

Program `14_01_docs_metadata.py` · sample `14_survey_raw.csv` (500 rows × 9 columns) · dictionary `14_data_dictionary.csv/.md`

[Code] Auto Data Dictionary and Documentation Check-up

```python
def field_dictionary(df, units):            # build one dictionary row per column
    rows = []                               # one row per field
    for col in df.columns:                  # loop over all the columns
        s = df[col]                         # take the column
        if pd.api.types.is_numeric_dtype(s): # numeric: give the range
            rng_txt = f"{s.min():.2f} ~ {s.max():.2f}" # minimum to maximum
        elif pd.api.types.is_bool_dtype(s):  # boolean: give the true rate
            rng_txt = f"{s.mean():.1%} true"  # share of True values
        else:                               # categorical: count and example
            rng_txt = f"{s.nunique()} categories" # number of categories
        rows.append({"field": col, "dtype": str(s.dtype), # name and dtype
                     "non_null_rate": f"{s.notna().mean():.1%}", # completeness
                     "n_unique": int(s.nunique()), # number of unique values
                     "range_or_sample": rng_txt,   # value-domain note
                     "unit": units.get(col, "-")}) # unit (if known)
    return pd.DataFrame(rows)               # assemble into a table
dictionary = field_dictionary(survey, UNITS) # build the field dictionary
dump(dictionary, "l4_data_dictionary.csv")  # write it to disk as CSV
```

The same dictionary is exported both as CSV and as Markdown and can go straight into the report; full program in `website/code/lecture04/l4_01_docs_metadata.py`.

Documentation and Data Contracts

Who does what

- Documentation: read by people – background, method, definitions, licence, version history;
- Contract: read by machines – types, value domains, missing conventions, primary-key uniqueness (already covered in L3);
- the two must be co-sourced: when the contract changes the documentation must follow, and the other way round.

Versions and checksums

Record the version (timestamp + a distinct ID) and a checksum: the first makes "which version?" traceable, the second makes "has the file been changed?" detectable. Together with L3's immutable snapshots this gives a complete chain of evidence.

Practical advice: write the checksum into the release script's log (`sha256sum`) and put the "data version number" into the caption of every chart.

Source: The version and checksum requirements follow standard data-management practice; the link to the L3 contract system is compiled for this lecture.

Documentation Acceptance Criteria

Four criteria

1. Understandable: an outsider can say "what this data is and on what definition";
2. Traceable: every field has a source and a processing record;
3. Verifiable: version numbers and checksums prove the file has not been altered;
4. In sync: when the data changes, the dictionary and the contract change with it.

Self-check before Part 2

- which of the 20 items does my `README` have?
- do I have an automatically generated field dictionary?
- are the units and the missing conventions written down?

Next part (frames 17–30): univariate descriptive statistics – the centre, dispersion and shape trio.

Descriptive vs Inferential

Definition

Descriptive statistics (count noun) are summary statistics that describe or summarise a set of information quantitatively; descriptive statistics (mass noun) is the process of using and analysing them. It differs from inferential statistics in its goal: to summarise the sample, rather than to use the sample to understand the population it represents.

Common measures

- Central tendency: mean, median, mode;
- Dispersion: standard deviation (variance), minimum and maximum, kurtosis, skewness.

Three uses in a report: (1) health check – is the data plausible (outliers, skew, bimodality)? (2) reporting – summarise the situation with a few numbers; (3) before modelling – decide whether to take logs, whether to standardise.

Describe first, infer later.

Source: The distinction between descriptive and inferential statistics follows standard statistical definitions; the three uses are compiled for this lecture.

Measures of Central Tendency

Key points

Central tendency looks at the average or middle value of the data, while dispersion looks at how spread out the data is; both kinds of measure help people understand the meaning of the data being analysed, with the help of figures, tables and discussion. Central tendency describes the centre of a distribution: the analyst looks at how often each data point occurs and summarises the most common pattern with the mean, median or mode.

Which one to use

- Mean: best when the distribution is symmetric with no extreme values, and easy to work with algebraically;
- Median: more representative of a "typical" value when the data is right-skewed or has outliers;
- Mode: for categorical data, or when you want the "most common value";
- Reporting advice: give the mean and the median together – the gap itself reveals the skew.

An intuitive example: a mean of 65 out of 100 sounds "middling", but the data may contain both 1 and 100 – which is exactly why we need measures of dispersion (next frame).

Source: The definitions of central tendency and dispersion follow standard statistical wording; the selection rules are compiled for this lecture.

Measures of Variability

Key points

Dispersion (or "spread") tells you how widely a set of data is spread: central tendency gives an average, but it does not say how the data is distributed within the set. The range, quartiles, absolute deviations and variance are all examples of dispersion measures.

A classic numerical example

Data: 5, 19, 24, 62, 91, 100; range $= 100 - 5 = 95$ (maximum minus minimum).

The temperament of four measures

- Range: looks only at the two ends, most sensitive to outliers;
- IQR (Q3−Q1): robust, used with box plots;
- MAD (median absolute deviation): a robust "average deviation";
- Variance / standard deviation: good algebraic properties and decomposable, but inflated by outliers.

Source: The definition of dispersion and the classic numerical example (range 95); the comparison of the four measures is on the sample pages (frames 23–24).

Distribution and Frequency Distribution

Key points

A distribution (or frequency distribution) is the number of times a data point occurs, or equivalently a measure of how often a data point does not occur. Example: the gender sequence male, male, female, female, female, other has the distribution – 2 male, 3 female, 1 other, 4 not male.

Three easily confused concepts

- Frequency: how many times (male = 2);
- Proportion: frequency divided by the total (male 33.3%);
- Density: frequency per unit interval of a continuous variable (the vertical axis of a histogram).

The commonest plotting mistake: mixing frequency and density on the vertical axis, or using a pie chart for a complement such as "not male" – a complement is not a category on the same level.

Source: The frequency-distribution explanation and the gender-counting example; the three-concept distinction and the plotting reminder are compiled for this lecture.

Skewness

Formula

Skewness (sometimes called relative skewness) is computed as the average of the cubed deviations from the mean, divided by the cube of the standard deviation to remove the unit. On this measure: a symmetric distribution has skewness 0, right-skew is positive, left-skew is negative.

How to read it

- Right skew (income, house prices, trading volume): mean $>$ median, long tail to the right;
- Left skew (a ceiling effect in exam marks): mean $<$ median;
- Rule of thumb: $|skew| > 1$ is clearly skewed, so consider logs or quantile descriptions.

This lecture's samples: city GDP and monthly income are both typically right-skewed – so the two sample pages that follow report both the mean and the median.

Source: The standard-deviation standardisation of skewness; the rule of thumb and the link to the samples are compiled for this lecture.

Kurtosis

Key points

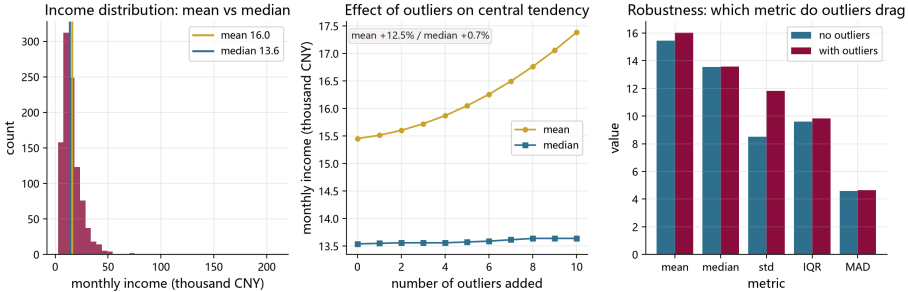
Kurtosis measures the weight of the tails relative to the rest of a distribution – that is, the total probability falling more than 2.5 standard deviations from the mean. Tails heavier than the normal distribution are called leptokurtic (fat-tailed), thinner ones platykurtic (thin-tailed), and those close to the normal mesokurtic. Fat-tailed (thin-tailed) distributions produce extreme deviations more (less) often than the normal.

Why it matters

- fat tails mean extreme events are more common than the normal assumption implies;
- financial returns, catastrophe losses and network traffic are all fat-tailed;
- risk management built on a normal assumption systematically understates extreme risk (link to L13).

Source: The kurtosis definition and the leptokurtic / platykurtic / mesokurtic naming; the risk-management implications are compiled for this lecture.

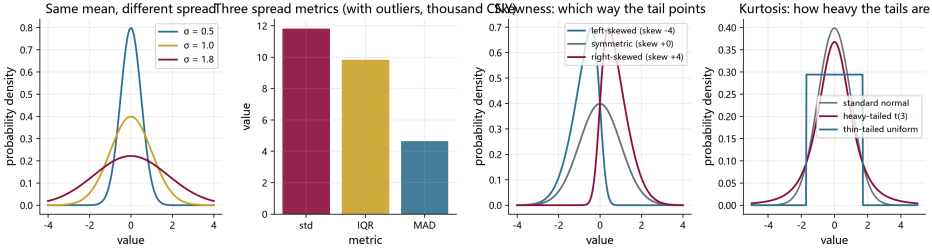
[Sample + Visualisation] Robustness of the Mean



1000 income observations + 3 high-income outliers: adding the outliers one by one, the mean moves $+12.5\%$ while the median moves only $+0.7\%$; the right panel compares how far each of five measures is pulled.

Program `14_02_descriptive_stats.py` · sample `14_income_skewed.csv` · table `14_descriptive_table.csv`

[Sample + Visualisation] Dispersion and Shape



First panel: the same mean, different dispersion; second: the three dispersion measures standard deviation / IQR / MAD; third: skewness (left-skewed / symmetric / right-skewed); fourth: kurtosis (fat-tailed $t(3)$ / normal / thin-tailed uniform).

Program `14_02_descriptive_stats.py` · one four-panel figure explains the two concept groups "dispersion" and "shape" at once

[Code] A Full Descriptive Pass

```
def describe_series(s):                     # compute the descriptive statistics
    return {                                # computed item by item
        "n": int(s.notna().sum()),          # valid sample size
        "mean": float(s.mean()),            # mean
        "median": float(s.median()),        # median
        "mode": float(s.round(1).mode().iloc[0]), # mode (binned to 0.1)
        "std": float(s.std(ddof=1)),        # standard deviation
        "range": float(s.max() - s.min()),  # range
        "iqr": float(s.quantile(.75) - s.quantile(.25)), # interquartile range
        "mad": float((s - s.median()).abs().median()), # median absolute deviation
        "skew": float(s.skew()),            # skewness
        "kurt": float(s.kurt()),            # excess kurtosis
    }                                       # end of the returned dictionary
rows.append({"sample": "with outlier", **describe_series(df["income_k"])}) # one summary row
table = pd.DataFrame(rows).round(3)         # three variants (clean / outlier / classic)
dump(table, "14_descriptive_table.csv")     # write the descriptive table to disk
```

Key point: pack the statistics into a function and reuse it on any dataset; the output goes straight to a file and joins the charts in the report. Full program `14_02_descriptive_stats.py`.

Five Univariate Charts

Five basic chart types

The features of a variable's distribution can be shown with figures or tables, including the histogram and the stem-and-leaf plot.

What each of the five does

- Histogram: the shape (skew, bimodality, long tails) – the bin count changes the impression;
- Box plot: quartiles and outliers, good for grouped comparison;
- Violin plot: box plot + kernel density, so multiple modes are visible;
- ECDF: quantiles and threshold proportions ("how many are below X");
- Stem-and-leaf: keeps the original values, good for reading small samples closely.

How to choose

To report the shape of a distribution → histogram or violin; to report "typical value and spread" → box plot; to report a threshold proportion → ECDF; for a very small sample ($n < 100$) → stem-and-leaf or a dot plot.

Source: The situations in which each of the five univariate chart types is used; selection advice is on the next frame's sample page.

Reading the Shape

Four signals

- Bimodality: probably two populations mixed together (two cities, two products) – stratify first, then look again;
- Long tail: extreme values or a power-law process; consider logs or quantile measures;
- Break / spike: an encoding problem (top-coding, rounding to integers);
- Zero inflation: a mass of zeros (no purchase, no click), which needs a two-stage model.

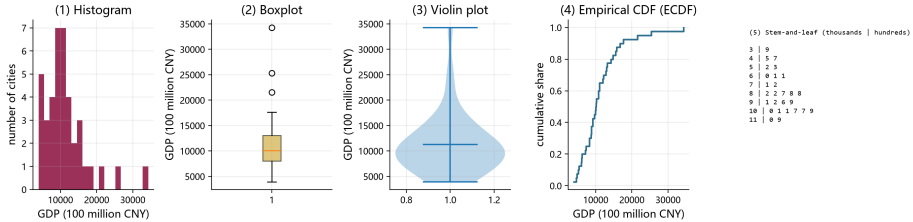
What to do about it

Signal	Response
Bimodal	stratify / facet, then redraw
Long tail	take logs, report the median
Spike	check the dictionary and the collection rules
Zero-inflated	model separately or binarise

Remember: a graphical anomaly is often a clue to a data problem – check the data first and change the model second, which links directly to the L3 cleaning workflow.

Source: The signal-to-response mapping is compiled from descriptive-statistics practice; the link to the L3 cleaning workflow is added for this lecture.

[Sample + Visualisation] One Variable, Five Charts



One column, city GDP: histogram (the bins decide the impression), box plot (outliers), violin plot (density shape), ECDF (threshold proportions), stem-and-leaf (original values). Changing the chart type changes the viewpoint.

Program `14_05_chart_zoo.py` · sample `14_city_gdp.csv` (40 cities)

Univariate Pitfalls

- The mean hides bimodality: two cities mixed together leave the mean in "no man's land";
- Mean vs percentile: average wages can rise just because there are more high earners;
- Bin counts manipulate the impression: the same data tells two stories with 10 bins or 50;
- Missing values silently dropped: the mean holds only for "those who have a value", so extrapolating is biased;
- Units confused: CNY / ten-thousand CNY / thousand CNY mixed up makes the mean differ a thousandfold.

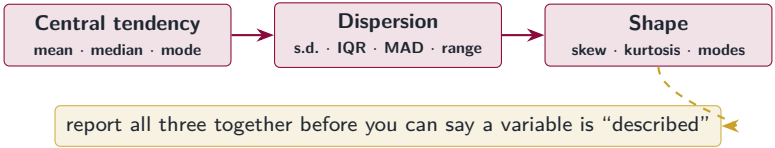
Three habits

1. pair every mean with a median;
2. when reporting a distribution give the sample size and the missing rate too;
3. put the binning parameters in the caption (`bins=30`, interval bounds).

Every later chart in this lecture follows these three rules: sample size, missing-value treatment, parameters go into the caption so that others can check your work.

Source: The pitfalls are compiled from descriptive-statistics practice and this lecture's samples (l4_01, l4_02).

The Descriptive Trio: Centre, Dispersion, Shape



Report template (one sentence)

"The variable has n valid observations (x missing), median m (mean $\bar{x}$), interquartile range IQR, skewness sk, and is right-skewed."

Next step

A single variable is only the starting point: real conclusions usually live in the relationship between two variables – so we move on to Part 3 (bivariate and multivariate).

Bivariate and Multivariate

Key points

When a sample contains several variables, descriptive statistics can describe relationships between variables. Here descriptive statistics include:

- cross-tabulations and contingency tables;
- graphical representation with scatter plots;
- quantitative measures of dependence;
- description of conditional distributions.

What each of the four is for

Tool	What it answers
Contingency table	are two categorical variables related?
Scatter plot	the shape of the relation between two numeric variables
Correlation	the strength of the relation (linear/monotone only)
Conditional distribution	the distribution when another variable is held fixed

Source: The four elements of multivariate description; the mapping to uses is compiled for this lecture.

Contingency Tables

How to read them (three tables in order)

1. Frequency table: how many people in each cell;
2. Row percentages: composition differences "within the same row";
3. Column percentages: origin differences "within the same column".

Frequencies alone show almost no relationship – only proportions are comparable.

Strength of association

- Chi-square test: tells you "independent or not" (only yes/no, not how strong);
- Cramér's V: a 0–1 strength measure (this lecture's sample gives a concrete value);
- Thresholds: 0.1 weak, 0.3 moderate, 0.5 strong (rules of thumb – judge in context).

This lecture's sample: the crosstab of income band $\times$ reach channel – the row percentages show that higher earners use online channels more often, but the association is weak, so the conclusion can only be "there is a difference", not "the channel determines income".

Source: Reading contingency tables follows general practice; Cramér's V and the rule-of-thumb thresholds are added for this lecture (code page l4_03).

Correlation: Pearson and Spearman

Measures

Quantitative measures of dependence include correlation – Pearson's r when both variables are continuous, and Spearman's ρ when one or both are not continuous – as well as covariance (which reflects the measurement scale of the variables).

Where each applies

- Pearson: a linear relationship with no extreme outliers; very sensitive to outliers;
- Spearman: only a monotone relationship is needed; robust to outliers and usable for ordinal variables;
- a large gap between the two usually means the relationship is non-linear or has outliers.

The most commonly misread sentence: "zero correlation does not mean no relationship" – a parabolic or piecewise relationship can also give $r \approx 0$. Plot the scatter first, then talk about the correlation.

Source: The definitions of the correlation measures; the applicability notes and the "zero correlation" reminder are compiled for this lecture.

Covariance and Regression Slope

Two kinds of slope

In regression the slope also reflects the relationship between variables: the unstandardised slope is the number of units the outcome changes when the predictor changes by one unit; the standardised slope expresses the same change in standardised (z-score) units.

Two reminders

- Covariance depends on the units: switching from CNY to ten-thousand CNY shrinks the covariance 10 000-fold, while the correlation is unchanged;
- unstandardised slopes cannot be compared across variables: to ask "which variable matters more", use standardised slopes or elasticities.

An international-trade example: in a gravity model the slope of $\ln(\text{trade})$ on $\ln(\text{distance})$ reads directly as an elasticity, which is the specification this lecture's sample uses (a 1% increase in distance lowers trade by about 1.15%).

Source: The definitions of the slope and the standardised slope; the units reminder and the elasticity example are compiled for this lecture.

Log Transform: Making the Picture Symmetric

Recommendation

Highly skewed data is often transformed by taking logarithms: logs make the picture more symmetric and closer to normal, which makes it easier to interpret visually.

Three gains and one cost

- compresses the right tail, so extreme values no longer dominate the axis;
- the slope reads as an elasticity (per cent against per cent);
- multiplicative relationships become additive, which suits linear models;
- the cost: you can no longer read values in CNY directly, and zeros and negatives need $\log(1+x)$ and similar.

This lecture's samples: after taking logs, the skewness of city GDP and monthly income falls markedly – which is why macro and financial data are almost always "logged before plotting".

Source: The motivation for the log transform; the gains and the cost are compiled for this lecture (linking to the transformation section of L3).

How a Restricted Range Weakens Correlation

The phenomenon: truncation weakens

A classic demonstration: the Pearson/Spearman correlation between X and Y in two situations – one where the ranges of the two variables are unrestricted, and one where the range of X is restricted to the interval $(0, 1)$. The conclusion: truncating the range weakens the correlation coefficient.

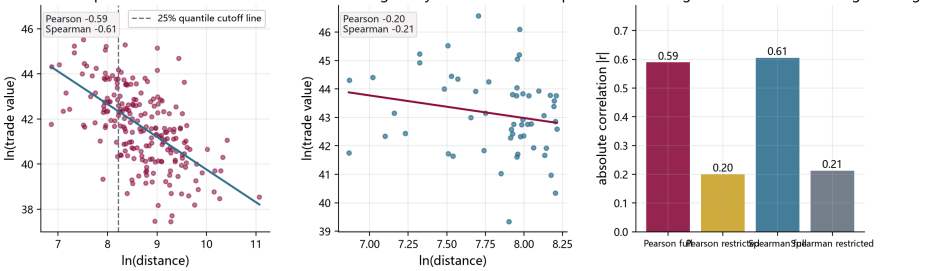
Why this happens

A correlation measures co-variation; when the values at one end are cut off, the "amount" of co-variation falls, and what remains is only a local slope – so r shrinks and its sign may even flip.

Practical implication: "I only studied a certain interval" must be declared up front; restricting the sample to "the nearest 25% of trading partners" and then reporting a very high correlation is a textbook case of selective reporting.

Source: The classic result that a restricted range weakens correlation; the mechanism and the practical implications are compiled for this lecture.

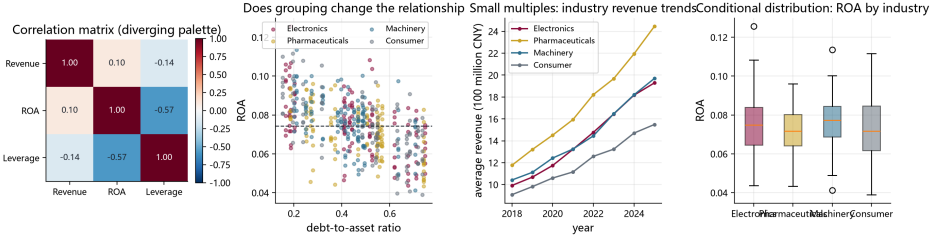
[Sample + Visualisation] Restricted Range in Action



Gravity-model data for 220 country pairs: the full sample gives Pearson $= -0.589$, Spearman $= -0.606$; keeping only the "nearest 25%" of partners drops these to -0.200 and -0.213 – only one third of the correlation strength remains.

Program `14_03_correlation_crosstab.py` · sample `14_trade_pairs.csv` · matrices `14_correlation_{pearson,spearman}.csv`

[Sample + Visualisation] Multivariate Views



60 firms × 8 years of panel data: the correlation matrix (diverging palette), a scatter coloured by industry (does grouping change the relationship?), revenue small multiples by industry, and the conditional distribution of ROA by industry (box plots).

Program `14_04_multivariate.py` · sample `14_firm_panel.csv` · conditional means `14_conditional_roa.csv`

[Code] Correlation, Crosstabs and Conditional Means

```
log_trade = np.log(pairs["trade_usd"])      # log: symmetrise a right-skewed variable
log_dist = np.log(pairs["distance_km"])     # log the distance as well
r_full = np.corrcoef(log_dist, log_trade)[0, 1] # Pearson correlation (full sample)
rho_full = pd.Series(log_dist).corr(pd.Series(log_trade), method="spearman") # Spearman
cut = np.quantile(log_dist, 0.25)           # 25% quantile as the cut point
mask = log_dist <= cut                      # truncation mask: nearest partners only
r_restricted = np.corrcoef(log_dist[mask], log_trade[mask])[0, 1] # Pearson after restriction
corr_pearson = analyze.corr(method="pearson").round(3)    # correlation matrix
ct = pd.crosstab(survey["income_band"], survey["channel"], # crosstab
                 margins=True, margins_name="Total")        # with marginal totals
ct_pct = pd.crosstab(survey["income_band"], survey["channel"], normalize="index") # row %: within-row composition
chi2, p, dof, expected = chi2_contingency(core) # chi-square test: are they independent?
cramers_v = float(np.sqrt(chi2 / (n_obs * (min(core.shape) - 1)))) # Cramer's V
```

One block gives both the full-sample and the restricted-range correlations, plus the crosstab + chi-square + strength measure: whether a conclusion is conditional is visible at a glance. Full program `14_03_correlation_crosstab.py`.

Grouped Comparison and Small Multiples

List of common chart types

Common types include: bar charts, distribution plots, weight trend charts, scatter plots, correlation matrices, heat maps, trend charts and flow charts.

How to group properly

- Small multiples: one panel per category, so shapes are comparable and do not overlap;
- Colour limit: at most 4–5 colours in one figure; beyond that, facet;
- Shared axes: the axis range must be the same across facets or the comparison is void;
- after grouping, put the sample size in the caption (small groups give jumpier lines).

A counter-example: 15 industries in one panel with 15 colours – the reader sees only a "ball of yarn". Facet when you should facet.

Source: The list of common descriptive chart types; the grouping advice is on the sample page (the small multiples on frame 38).

Conditional Distributions and Simpson's Paradox

What a conditional distribution is

The distribution of a target variable when another variable is held fixed. For example "the satisfaction distribution on the online channel" versus "the satisfaction distribution in stores" – you compare distributions, not just means (the means can be identical while the distributions are completely different).

Simpson's paradox

The overall relationship points the opposite way to the relationship within each subgroup: this happens when the grouping variable is unevenly distributed. The remedy: look at stratified charts first, then decide whether the grouping variable must be controlled for (links to L12 on causal inference).

Practical advice: every "overall correlation" comes with a grouped scatter or conditional box plot; if the relationship reverses after grouping, the report must give both levels of conclusion.

Source: Conditional distributions are one element of multivariate description; Simpson's paradox and the remedy are compiled for this lecture.

Three Questions about Correlation

The three questions

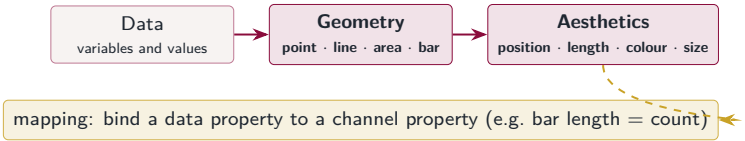
1. Direction: positive or negative? (look at the scatter and the sign of the coefficient)
2. Strength: how large is $|r|$? (judge in context, don't read "0.5 = strong")
3. Form: linear or monotone? is there grouping or truncation?

Two disciplines

- reporting a correlation always comes with a scatter plot (the lesson of Anscombe's four datasets);
- reporting a correlation always states the sample range (a restricted range weakens it).

Next part (frames 43–56): chart choice and the grammar of graphics – translating "relationships" into "graphics".

The Grammar of Graphics



Core principle

From an academic point of view, visualisation can be seen as the mapping of raw data (usually numeric) to graphical elements (lines or points in the figure); the mapping determines how the properties of those elements vary with the data. For example, a bar chart maps the length of the bar to the quantity of the variable. Because the graphical design of the mapping affects how readable the chart is, mapping is the core skill of data visualisation.

Same data, different mapping, different truth.

Source: The three-layer structure and the channel concept are the general framework of the grammar of graphics (Wilke, Munzner).

Which Channel Is Read Most Accurately

From most to least accurate

1. Position (points on a shared scale): most accurate;
2. Length (bars): very accurate;
3. Angle / slope: moderate (pie slices and line slopes both rely on it);
4. Area: less accurate (the eye estimates area from radius, and overestimates);
5. Volume / 3D: least accurate;
6. Colour lightness, hue: good for categories and order, not for precise comparison.

Practical implications

- to compare magnitudes → use position or length (bar chart, dot plot);
- don't use a pie chart to compare similar shares (a controlled experiment is on frame 79);
- bubble charts must encode by area, never by radius;
- colour is only a supporting channel, and needs redundant encoding.

Source: The channel-effectiveness ranking follows the general conclusions of perceptual research in visualisation (the Cleveland & McGill line of work); the practical implications are added for this lecture.

Choose the Chart by the Question

Type of question	First choice	Second choice / use with care
Comparing magnitudes	bar chart / dot plot (sorted)	radar chart (hard to compare); pie chart (only 2–3 categories)
Seeing a distribution	histogram / box plot / violin / ECDF	mean $\pm$ error bar (hides multimodality)
Seeing change	line chart (watch the baseline and smoothing)	stacked area (only for the overall trend)
Seeing relationships	scatter / correlation matrix (guard against overplotting)	dual-axis line chart (manufactures false correlation)
Seeing composition	stacked bar chart / Sankey diagram	pie and doughnut charts (angles are imprecise)
Seeing space	choropleth (rates) / symbol map (totals)	3D maps (misleading in both area and perspective)
Seeing structure	network diagram (few nodes, few colours)	large force-directed graph (unless it is interactive exploration)

How to use it: write your question sentence first ("which channels have higher satisfaction?"), then look it up in this table – the question decides the chart, not the other way round.

Source: The decision table is compiled from general chart-selection practice and covers the common descriptive chart types.

Comparison and Distribution

Comparison: columns and bars

- Sorting is the cheapest improvement: readers see the ranking instantly;
- vertical columns suit time; horizontal bars suit long labels;
- the vertical axis starts at 0 (the basic discipline of a bar chart, frame 75);
- more than 10 categories $\rightarrow$ take the top N plus "other".

Distribution: box and violin

- box plots suit grouped comparison (same axis, same scale);
- violin plots reveal multimodality but need enough observations;
- for small samples ($n < 20$) draw a dot plot, not a box plot;
- when reporting a distribution state n, the median and the quartiles.

"Bar chart / distribution chart" correspond precisely to these two kinds of question and are already in the chart-selection decision table above.

Source: The key points are compiled from the common chart types and the chart-selection decision table above.

Time: Trend and Weight Trend Charts

Two kinds of time chart

Weight trend charts and trend charts. The first is common for how portfolio weights change over time (finance); the second for how a metric itself changes over time.

Smoothing and noise

Moving averages and seasonal adjustment make a trend clearer, but must be stated in the caption: smoothing hides turning points and can also manufacture a "trend" (end-point effects).

How to choose the baseline

- index it (first period $= 100$) to compare different magnitudes;
- broken axes are forbidden (frame 75), but "starting at 0" is not mandatory for a line chart – a line compares rates of change, not lengths;

An impulse chart is the extreme event-type case: annotate the policy or event on the time axis so that readers can see "when the change happened".

Source: Time-chart types (trend, weight trend, impulse chart); the baseline and smoothing reminders are compiled for this lecture.

Relationships: Scatter, Matrix and Heat Map

The number-one scatter problem: overplotting

Too many points smear into a blob. Fixes: adjust opacity, jitter, sample, switch to a 2D density or hexagonal binning (hexbin), or draw quantile curves directly.

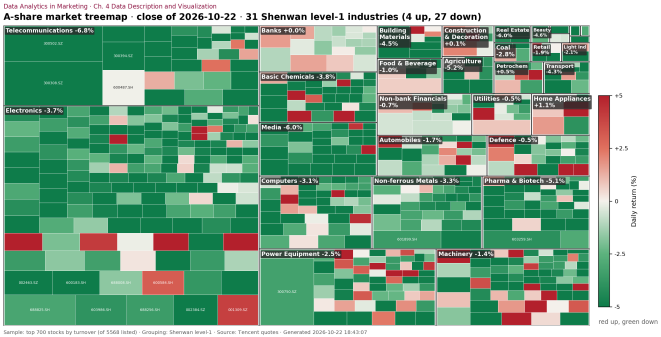
Using matrices and heat maps

- correlation matrix: diverging palette + numeric labels (used in this lecture's samples);
- the upper triangle repeats, so half is enough;
- cluster reordering (a clustered heat map) reveals "blocks of variables";
- heat maps must not use a rainbow palette (the sense of order breaks down).

"Scatter plot, correlation matrix and heat map" correspond to exactly these three things; this lecture draws all of them at once on the firm panel data (sample page, frame 38).

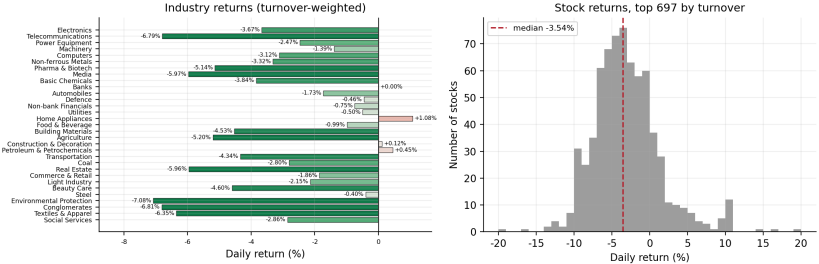
Source: Relationship chart types (scatter, correlation matrix, heat map); the overplotting fixes and the palette advice follow visualisation practice.

[Sample + Visualisation] Today's A-Share Treemap



One chart, three channels: position = industry, area = turnover, colour = daily return. On 22 October 2026, across 31 Shenwan level-1 industries, only four rose; the median stock lost -3.54% against -1.48% for the index – large caps held the index up while the typical stock fell much further.

[Sample + Visualisation] Same Data, Two Readings: Treemap and Distribution



Left: the weighted return of the 31 industries (exactly the average colour of each tile in the previous treemap); right: the distribution of returns across the 697 stocks – very thin around zero, massed near the median of -3.5%. One dataset, and the treemap shows structure and magnitude while the histogram shows distribution and tails; the "choose the chart by the question" rule of frame 44 is one sentence here: ask "which industries fell most" and use the treemap; ask "how widespread were the falls" and use the distribution.

Two charts from one data pack (`heatmap_industry.json`); the distribution is produced by `render_heatmap.py` with the same red–green diverging palette as the treemap

Structure and Flow: Flow Charts and Networks

Two kinds of structure chart

Flow charts (describing an analysis pipeline or a supply-chain route) and network charts (Gephi for graph and network visualisation, CiteSpace for citation structure, SNAP for patent citation graphs).

Four rules for a good network chart

1. keep the node count to a few dozen, or aggregate or filter first;
2. state the layout (force-directed / circular / geographic) and fix the random seed;
3. encode different meanings in size and colour (degree, community) and give a legend;
4. hand large graphs to interactive tools (Gephi/CiteSpace) to explore, and put a simplified version in the report.

Supply chains, trade networks and citation networks all read the same way: first the hubs, then the communities, then the direction.

Source: Structure and network chart types and common external tools; the four rules are compiled for this lecture (the trade-network case is on frame 74).

Event Narrative: Impulse Charts and Time-axis Annotation

Impulse chart

An impulse chart puts events and changes in a metric on the same time axis: the metric varies over time, the events appear as vertical lines or markers, and it answers "when did the change happen and what might have caused it".

Three points

- events must be genuine and verifiable (a policy release date, an earnings date), with the source stated;
- annotations must not cover the data (put them outside the plot or use leader lines);
- do not imply causation: being adjacent in time is not causation (links to L12).

Both the energy and the experiment samples in this lecture can carry event annotations – for example "the billing method changed from this month" – and such charts are common in international-trade and financial post-mortems.

Source: Impulse charts (event-type time charts); the key points and the causation reminder are compiled for this lecture.

Figure and Table Conventions

Numbering and placement

Figures and tables in the text should be as simple as possible; number them Figure 1, Figure 2, ... or Table 1, Table 2, ..., and place them as close as possible to the text that first refers to them.

Self-contained captions (emphasised here)

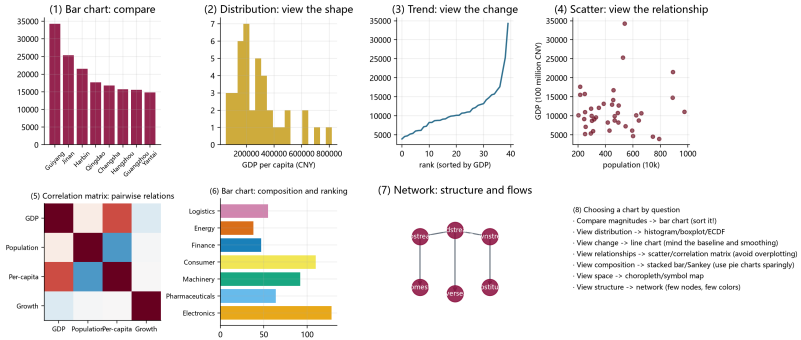
A good caption must answer:

- what the data is (source and file);
- what n is and how missing values were handled;
- the definition and the units; whether the axis is truncated;
- the uncertainty (error bars/interval) and the statistical method.

An example caption (the full template is on slide 88): "Figure 1 Mean satisfaction by channel (data l4_survey_raw.csv, $n = 500$; satisfaction 1–5, 5 missing not imputed; the vertical axis starts at 2.5, only to show differences between channels)."

Source: Figure and table numbering and in-text placement follow general convention; the four elements of a self-contained caption are compiled for this lecture.

[Sample + Visualisation] The Chart Zoo



(1) bar chart (comparison) (2) distribution (3) trend (4) scatter (relationships) (5) correlation matrix (6) bar chart (composition) (7) network chart (structure), and the eighth panel gives the choose-the-chart-by-question mnemonic – covering the list of common chart types.

Program `14_05_chart_zoo.py` · samples `14_city_gdp.csv`, `14_category_counts.csv`

[Code] A Chart Factory: One Style, Batch Output

```python
def style_axes(ax, title=None, xlabel=None, ylabel=None): # one place to set title and labels
    if title:                                  # set the title only if given
        ax.set_title(title)                    # subplot title
    if xlabel:                                 # set the x label if given
        ax.set_xlabel(xlabel)                  # x-axis label
    return ax                                  # return the same object for chaining
def annotated(ax, text, loc="upper left"):      # add an in-panel note
    xy = {"upper left": (0.02, 0.96), "lower right": (0.98, 0.06)}.get(loc, (0.02, 0.96)) # position lookup, default top-left
    ax.text(xy[0], xy[1], text, transform=ax.transAxes, va="top", ha="left", # place text in axes coords
            fontsize=8.6, bbox=dict(boxstyle="round,pad=0.25", fc="#F7F3F2"))
    return ax                                  # return the same object
fig, axes = plt.subplots(2, 4, figsize=(13.8, 6.1)) # two rows, four columns, eight panels
axes[0, 0].bar(top["city"], top["gdp_yi"], color="#8A0C3C") # bar chart: compare magnitudes
axes[0, 1].hist(cities["gdp_per_cap"], bins=18)     # distribution
axes[0, 2].plot(range(len(trend)), trend, "-")      # trend
axes[0, 3].scatter(cities["pop_wan"], cities["gdp_yi"], s=22) # scatter
```

Once titles, axis labels and in-panel notes are wrapped in functions, each chart needs only one or two lines – the style is consistent by construction and font sizes and palettes can be changed in one place. Full program `14_05_chart_zoo.py`.

Axes, Scales and Reference Lines

Four rules

1. a bar chart's vertical axis must start at 0 (length encoding);
2. a line chart may start elsewhere, but this must be declared in the caption with a visual cue (such as a break symbol);
3. log axes are for data spanning orders of magnitude; the ticks must show the real values (1, 10, 100);
4. avoid dual axes: when two units share one chart, the visual correlation is drawn, not in the data.

Ticks and gridlines

- 4–8 ticks is about right, at rounded intervals (multiples of 1, 2, 5);
- gridlines should be light and serve reading values;
- reference lines (zero, target, mean) must state their meaning explicitly.

Source: The rules are compiled from general chart-design practice; the dual-axis risk links directly to the misleading-chart comparison on frame 77.

Colour: Continuous, Categorical and Diverging

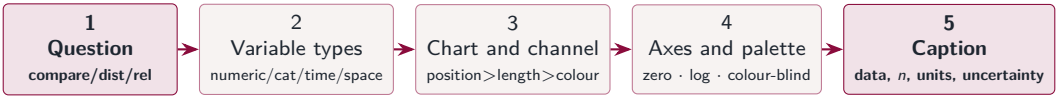
Palette type	Data it suits	Examples and cautions
Continuous / sequential	continuous variables with an order (GDP, density, correlation strength)	viridis, RdPu; light→dark must change lightness monotonically
Categorical	unordered categories (industry, country, channel)	Okabe–Ito, ColorBrewer; at most 5–8 colours
Diverging	differences with a central reference (growth rates, deviation from the mean)	RdBu; the centre must be the "zero/baseline" value

Three disciplines: (1) colour does not carry precise comparison (use position/length); (2) categorical palettes must be colour-blind friendly (frame 84); (3) in one report, the same meaning gets the same colour – don't let red mean "loss" on one slide and "China" on the next.

Colour is a channel, not decoration.

Source: The palette taxonomy and examples follow ColorBrewer and the matplotlib documentation; the disciplines are added for this lecture.

Choosing a Chart in Five Steps



Self-check

Show the chart to a classmate for 10 seconds. Can they state your conclusion? If not, the problem is usually in step 1 or step 3.

Next step

Next part (frames 57–74): building these charts in Python – interactive charts, dashboards and geo-visualisation.

What Data Visualisation Is

Definition

Data and information visualisation is an interdisciplinary field concerned with the graphical representation of data and information. When the amount of data or information is large (time series, for example) it is a particularly efficient way to communicate. It is also the study of the visual representation of abstract data to augment human cognition; abstract data includes both numeric and non-numeric data (such as text and geographic information).

Why it works

The human eye resolves position and length far faster than it resolves numbers: a good chart turns "comparison" from "reading numbers" into "seeing shape", leaving attention for the conclusion rather than the lookup.

Source: The definition of visualisation follows standard wording in visualisation research.

Info-viz vs Sci-viz

The distinction

When the spatial representation is chosen (a graphic-design page layout, for example) we call it information visualisation; when the spatial representation is given (real physical space, for example) we call it scientific visualisation. It is related to both infographics and scientific visualisation.

In this course

- Information visualisation: bar charts, heat maps, network charts – we decide the layout;
- Scientific visualisation: maps, meteorological fields, medical images – reality fixes the coordinates;
- the map section of this lecture (frames 67–73) belongs to the latter: the correctness of location is non-negotiable.

Source: The distinction between information and scientific visualisation follows standard wording in visualisation research; the mapping to the map section is compiled for this lecture.

Tools Overview: Everything in Python

Tool	Strengths	How this lecture uses it
Stata	strong econometrics; standardised graphics, no maps or networks	command comparisons are kept, but all work is done in Python
Tableau	drag-and-drop dashboards, interactive filtering	reproduced with plotly subplots + HTML (frames 65–66)
R	the ggplot2 grammar of graphics, a rich statistics-graphics ecosystem	used as a reference: the grammar-of-graphics idea is shared with Python
Python	the fullest ecosystem, programmable and reproducible	all the examples here: matplotlib / seaborn / plotly

Three reasons to standardise on Python: (1) the same environment as the L3 preparation scripts, so one pipeline runs end to end; (2) charts are reproducible (scripted, fixed seeds); (3) no licence and no installation of commercial software – students can do it on their own laptops.

Source: Tool positioning follows each tool's official documentation; "everything in Python" is fixed by this semester's teaching policy (no Power BI / Tableau).

Stata: Strengths and Limits

Strengths and limits

- Strengths: focused on econometric analysis;
- Weak points: fairly standard graphics, no map or network tooling;
- common commands: `twoway`, `matrix`, `bar`, `box`, `dot`, `pie`.

The lesson

A tool's limits decide what your chart can express: when you need maps, networks or interactivity, Stata's weak points show – this is not about "which software is better" but about matching the task to the tool. The map and interactive examples in this lecture exist to fill exactly those two gaps.

If you go on to do econometric analysis, Stata remains useful; but exporting results and plotting them in Python is common practice: each does its own job, and the data definitions must still be consistent.

Source: Stata's strengths and common commands; the lesson and the division of labour are compiled for this lecture.

The Python Stack

Library	Positioning	Where it appears here
matplotlib	low-level control: everything is adjustable, good for final artwork	the workhorse for the 13 sample figures (frame 53)
seaborn	a fast statistical-graphics interface: distributions, groups, regression	distribution and grouped comparisons
plotly	interactive charts and maps: hover, zoom, export to HTML	interactive charts, dashboards, choropleth (frames 63–70)
altair	a declarative grammar (Vega-Lite), good for standardised output	an extension, to see the grammar of graphics

Recommended resources: `python-graph-gallery.com` (find code by chart type) and Kaggle Learn's Data Visualization course; on the R side, `data-to-viz.com` and `r-graph-gallery.com`.

Source: The library positioning and the mapping to this lecture are compiled for this lecture; resource links are on each site's home page.

The R Ecosystem and Learning by Chart Type

The ggplot2 grammar of graphics

Combine layers with `ggplot(data, aes(...)) + geom_*()`: the "data–channel–geometry" grammar written as code. This is exactly the grammar of graphics from frame 43 – learn one and the other follows naturally.

How to use the two sites

- `data-to-viz.com`: recommends charts by data type and question;
- `r-graph-gallery.com` / `python-graph-gallery.com`: find copyable code by chart type;
- Kaggle Learn: interactive exercises.

Learning advice: do not start from a library's API, start from "what do I want to express" and search for the chart type – the chart-selection decision table in this lecture (frame 45) is that list of search keywords.

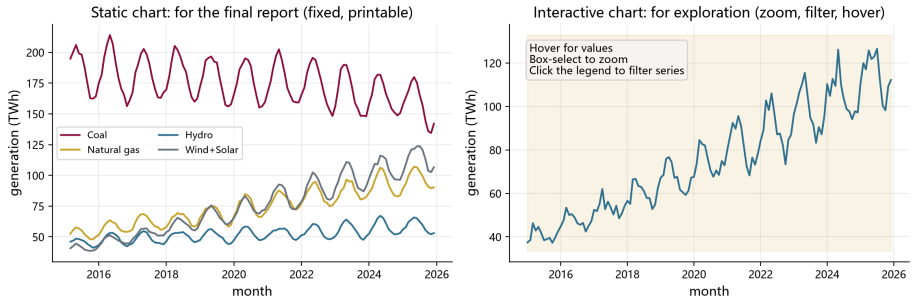
Source: The resources and their use follow each site's official description; the learning advice is compiled for this lecture.

[Code] Interactive Charts with plotly

```
line_fig = px.line(energy, x="month", y="generation_twh", color="source", # line: colour by source
                   title="Monthly generation (hover for values, box-zoom)", # title states what is interactive
                   labels={"month": "Month", "generation_twh": "Generation (TWh)"}) # axis labels
line_fig.write_html(INT / "energy_trend.html", include_plotlyjs="cdn") # export a single HTML file
plot_df = survey.dropna(subset=["satisfaction", "income_k", "spend_k"]) # plotly rejects missing values
scatter_fig = px.scatter(plot_df, x="income_k", y="spend_k", color="channel", # scatter: colour by channel
                         size="satisfaction", hover_data=["city", "education"]) # show detail on hover
scatter_fig.write_html(INT / "income_spend.html", include_plotlyjs="cdn") # export the interactive scatter
sankey_fig = go.Figure(data=[go.Sankey(                # Sankey: composition and flow
    node=dict(label=labels, pad=14, thickness=16),     # node style
    link=dict(source=list(range(len(flow))),           # link start points
              target=[len(flow) + i for i in range(len(flow))], # link end points
              value=flow["generation_twh"].round(1).tolist()))]) # link width = value
sankey_fig.write_html(INT / "energy_sankey.html", include_plotlyjs="cdn") # export the Sankey
```

Two things to watch with interactive charts: (1) handle missing values first (plotly raises an error); (2) `include_plotlyjs="cdn"` keeps the HTML to a few tens of KB, easy to ship with the repository. Full program `l4_06_plotly_interactive.py`.

[Sample + Visualisation] Interactive vs Static



Left: static charts – the information is fixed, printable and report-ready; right: interactive charts – zoom, filter and hover for values, good for exploration and classroom demonstration. The repository ships 6 interactive HTML files you can open (energy trend, income scatter, Sankey, world map, flow map, dashboard).

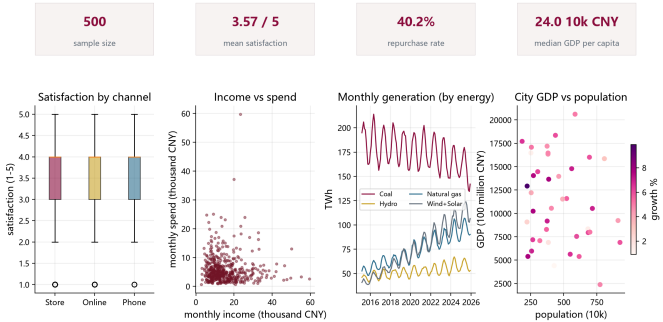
Program `14_06_plotly_interactive.py` · interactive files `figures/lecture04/interactive/` · sample `14_energy.csv`

[Code] A Python Dashboard (Instead of a BI Tool)

```python
kpi = {"sample size": f"{len(survey):,}",       # top KPI card: sample size
      "mean satisfaction": f"{survey['satisfaction'].mean():.2f} / 5", # mean satisfaction
      "revisit rate": f"{survey['revisit'].mean():.1%}", # share who come back
      "median GDP per capita": f"{cities['gdp_per_cap'].median()/1e4:.1f} (10k CNY)"} # city median
fig = make_subplots(rows=2, cols=2,             # 2x2 panel layout
                    subplot_titles=("Satisfaction by channel", "Income and spend", # four subplot titles
                                    "Monthly generation (by source)", "City GDP and population")) # titles continue
fig.add_trace(go.Box(y=sub, name=ch, boxmean=True), row=1, col=1) # top left: grouped box plot
fig.add_trace(go.Scatter(x=survey["income_k"], y=survey["spend_k"], # top right: scatter
                         mode="markers", name="respondent"), row=1, col=2) # placed top right
fig.add_trace(go.Scatter(x=sub["month"], y=sub["generation_twh"], # bottom left: time series
                         mode="lines", name=src), row=2, col=1) # placed bottom left
fig.add_trace(go.Scatter(x=cities["pop_wan"], y=cities["gdp_yi"], # bottom right: city metrics
                         text=cities["city"], mode="markers"), row=2, col=2) # placed bottom right
fig.write_html(INT / "dashboard.html", include_plotlyjs="cdn") # export a single-file HTML dashboard
```

The same data and the same definitions produce both an interactive HTML dashboard and a static figure (frame 66); no server and no licence – copy the file to someone and they can double-click it. Full program `14_07_dashboard.py`.

Dashboard Design Principles



Four KPI cards at the top (the conclusion at a glance) + a 2×2 panel (distribution, relationship, trend, geographic metric): answer four questions on one screen, rather than stacking twenty charts together.

Layout hierarchy: conclusion (KPIs) → key comparisons → detail; next step: add interactive filtering with Streamlit or Dash (this lecture needs no server, using single-file HTML).

Structure of Geospatial Data

Three common forms

- Point data: latitude and longitude (stores, ports, events);
- Polygon data: administrative boundaries (GeoJSON / Shapefile), used for choropleth maps;
- Raster data: satellite imagery, meteorological fields, a matrix of pixels.

Two things you must know

- Coordinate reference system (CRS): `EPSG:4326` is latitude/longitude (degrees), `EPSG:3857` is Web Mercator (metres); a projection changes areas and shapes;
- Geocoding: address → latitude/longitude, whose quality depends on the address database and the matching strategy.

How this lecture does it: province boundaries come from a public GeoJSON (downloaded and cached into `data/lecture04/` at build time), and point data comes straight from latitude/longitude columns; if the boundary file is unavailable the code falls back to a centroid bubble map so the pipeline never breaks.

Source: Geospatial data structures and CRS are general GIS knowledge; the data and the fallback strategy here are on code page 69.

Choropleth Maps and Classification Methods

Three classifications

- Equal interval: split the value range evenly – extreme values "eat up" all the colour;
- Quantile: the same number of units per class – strong visual contrast, but differences within a class are hidden;
- Natural breaks (Jenks): minimise within-class variance and maximise between-class variance – closest to the data's own distribution.

Three disciplines for a choropleth

1. it can only express a rate or intensity (per capita, share, density), never a total (large areas win automatically);
2. state the classification method and the colour scale – change the classification and the map changes;
3. areas with no data are drawn grey and explained, never treated as 0.

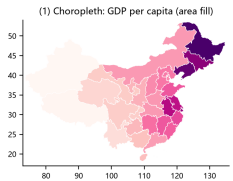
Source: Classification methods and disciplines follow general cartography and data-journalism practice; the examples here are on sample page 70 and code page 69.

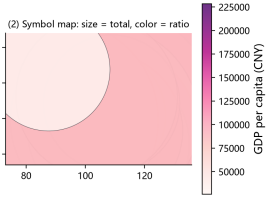
[Code] Three Maps: Choropleth, Symbol Map and Flow Map

```
value = prov.set_index("province")["gdp_pc"]  # GDP per capita indexed by province name
if geojson:                                   # boundary file present: draw a real choropleth
    for feat in geojson["features"]:          # loop over every province feature
        short = feat["properties"]["name"].replace("省", "").replace("市", "") # strip the Chinese suffix first
        ring = np.array(feat["geometry"]["coordinates"][0]) # take the outer ring coordinates
        patches.append(Polygon(ring, closed=True)) # build the polygon
        val = float(value[short])             # GDP per capita for that province
        colors.append(cmap((val - vmin) / (vmax - vmin + 1e-9))) # normalise, then pick a colour
    axes[0].add_collection(PatchCollection(patches, facecolor=colors)) # draw them in one batch
axes[1].scatter(prov["lon"], prov["lat"], s=prov["gdp_yi"] * 3.2, # symbol map: size = total
                c=prov["gdp_pc"], cmap=SEQ)               # colour = per capita
fig_flow.add_trace(go.Scattergeo(lat=links["lat"], lon=links["lon"], # flow map: destination bubbles
    marker=dict(size=links["trade_share"] * 2.2, color=links["trade_share"]))) # bubble size and colour = trade share
world = px.choropleth(cty, locations="iso3", color="gdp_pc", # world choropleth
                      locationmode="ISO-3", color_continuous_scale="RdPu") # ISO-3 lookup + continuous palette
```

Two charts from the same province data: the choropleth shows rates and the symbol map shows totals; the world map is located by ISO-3 code (no geometry file needed). Full program `14_08_geo_maps.py`.

[Sample + Visualisation] Choropleth vs Symbol Map





Which one and when?

- To express a ratio/intensity (per capita, share, density) -> choropleth
  but large areas dominate (area = visual weight)
- To express a total (GDP, population, trade value) -> symbol map
  avoid letting a large province with a small population distort the conclusion
- To express connections (trade, migration) -> flow map / network
- Classification: equal interval vs quantile vs natural breaks
  change the classification and the color distribution changes -- put it in the caption

Left: choropleth – filled by province-level GDP per capita (boundaries from a public GeoJSON, 35 features); middle: symbol map – bubble size = total GDP, colour = per capita; right: how to choose between the two maps, and a classification reminder.

Program `14_08_geo_maps.py` · sample `14_province_gdp.csv` · interactive maps `interactive/map_world_choropleth.html`, `map_trade_flows.html`

Spatial Autocorrelation: Moran's I

Three concepts

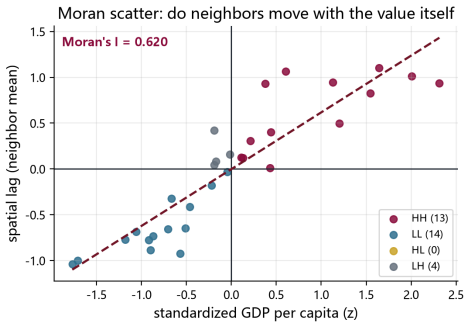
- Spatial weights matrix W: defines "who is whose neighbour" (contiguity or K nearest neighbours), row-standardised;
- Spatial lag: Wz, the average of each region's neighbours;
- Moran's I: $I = \frac{\sum_i z_i (Wz)_i}{\sum_i z_i^2}$, taking values roughly between -1 and 1.

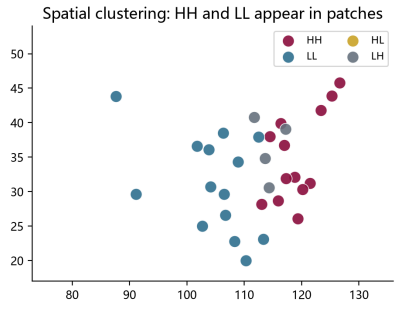
How to read it

- $I > 0$: clustering (high values cluster together, low values cluster together);
- $I \approx 0$: a random pattern;
- $I < 0$: dispersion (high and low alternate, like a chessboard);
- the four quadrants: HH (high–high), LL (low–low), HL, LH; the last two are outliers.

Source: The classic statements of Moran's I and LISA (Anselin); the implementation of the weights matrix and the spatial lag is on code page 69.

[Sample + Visualisation] Moran Scatter





31 province-level units with a 4-nearest-neighbour weight: Moran's I $= 0.659$, quadrant counts HH 16, LL 13, LH 2 – high values cluster in the east and low values in the west, so GDP per capita shows clear positive spatial clustering.

Program `14_08_geo_maps.py` · sample `14_province_gdp.csv` · results `14_moran.csv`

Four Map Pitfalls

Pitfall	The problem	What to do
Distorted area	large, sparsely populated regions dominate visually	use rates, not totals; where necessary use a cartogram or a grid map
Classification manipulation	change the classification and the conclusion changes	state the classification method; also give a continuous scale or the class intervals
Projection choice	different projections change areas and shapes	label the projection; use an equal-area projection for small regions
MAUP	the modifiable areal unit problem: different aggregation levels give different correlations	state the spatial unit; run a multi-scale robustness check where necessary

One reminder: a map is the strongest persuasive tool and also the easiest to manipulate – every map must state "which indicator, which classification, which projection, and which regions are missing".

Source: The four pitfalls are compiled from spatial statistics and data-journalism practice; MAUP is a classic problem in geography (Openshaw).

Networks, Citations and Trade Flows

Common resources

- WITS (World Bank) international-trade visualisation: exporter–buyer network diagrams, filterable by HS code, year and threshold (`wits.worldbank.org`);
- Gephi: an open-source platform for exploring graphs and networks;
- CiteSpace: interactive visualisation of citation structure and temporal patterns;
- SNAP (Stanford): datasets of patent and citation networks.

Three questions to ask of a network

1. who are the hubs? (largest degree / centrality)
2. how do communities split? (colour by cluster)
3. direction and weight? (what do the arrows and line widths represent?)

A reproducible substitute for this lecture: draw the flow map and the Sankey diagram in Python on synthetic trade data (`interactive/map_trade_flows.html`), and use WITS's real network graph in class as a comparison.

Source: Resource list (WITS, Gephi, CiteSpace, SNAP); the three network questions and the Python substitute are compiled for this lecture.

Misleading (1): Truncated Axes

The trick

Change a bar chart's vertical axis to start at "minimum -1" instead of 0: the same data makes the gap between group A at 11.1% and group B at 13.1% look several times larger.

Why it misleads

A bar chart encodes values with length; once truncated, length is no longer proportional, and the reader's "twice as tall" intuition has nothing to do with the data – the channel has been broken.

The fix

- a bar chart's vertical axis always starts at 0;
- a line chart may zoom into a range, but this must be declared in the caption with a break symbol;
- to emphasise a difference → use a difference chart (the confidence interval of B−A), not a compressed axis.

A controlled experiment from this lecture's sample: for the same A/B data, the truncated version looks like "group B is far ahead", while the version starting at 0 shows a difference of only 2 percentage points.

Source: Truncated axes are a classic misleading technique discussed in visualisation books such as *How Charts Lie*; the controlled experiment is on sample page 79.

Misleading (2): Area, Volume and 3D Illusions

Three classic errors

- encoding bubbles by radius: double the value, quadruple the area – a visual exaggeration;
- 3D pie charts: perspective makes the front slices look larger and distorts the angles;
- double encoding: using size and colour for the same value at once, "magnifying twice".

The fix

- bubbles by area (in code, $s \propto$ the value, not the radius after a $\sqrt{\ }$);
- consider a pie chart only for "parts of a whole" with ≤ 3 categories; use bars to compare similar shares;
- encode one value with one channel and leave the other channels for other variables.

Source: Area/volume illusions and the improved practice follow perceptual research in visualisation; the controlled experiment is in column (2) of sample page 79.

Misleading (3): Dual Axes and Cherry-picking

False correlation from dual axes

Plot "conversion rate" and "sample size" on the left and right vertical axes and the similarity of the two curves depends entirely on the two scales chosen – change a scale and they are "uncorrelated". The right approach: draw a scatter plot to see the real relationship and give the correlation coefficient.

Selective ranges

Plot only the first five days "when the treatment was ahead", or report only the favourable months or subsamples. The remedy: plot the whole range, mark the truncated part, and state the rule for choosing the range in the caption (whether it follows a pre-registration or a pre-specified analysis plan).

What these tricks have in common: the data is not faked, only the presentation is filtered. That is why this lecture stresses that "chart honesty" is part of research integrity.

Source: Dual axes and selective reporting are common issues in chart criticism and open-science discussion; the remedies are added for this lecture.

Misleading (4): Statistics Meets Graphics

- Simpson's paradox: the total rises while every group falls (frame 41);
- Survivorship bias: counting only the individuals that "survived", which inflates the mean;
- a log axis hides absolute gaps: the growth rates are equal while the absolute gap widens;
- no uncertainty reported: presenting "maybe" as "certainly" (frame 81).

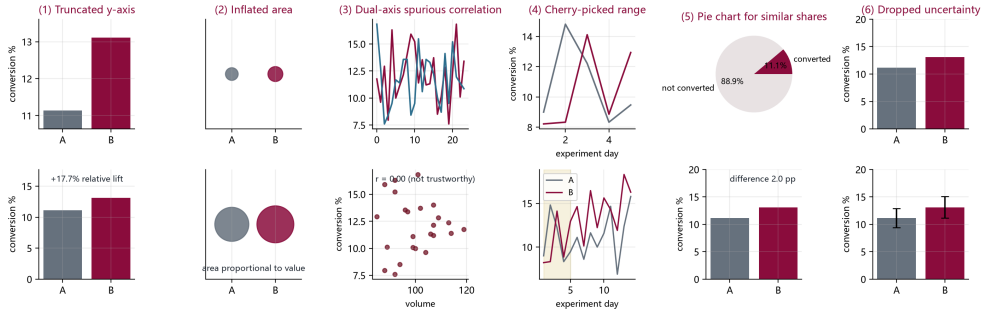
Three lines of defence

1. stratified re-check: run a grouped check on every overall conclusion;
2. give relative and absolute together (per cent and percentage points);
3. interval + sample size: let readers judge the strength of the evidence themselves.

Remember one thing: charts do not lie by themselves, the people who choose them do – the checklists in this lecture (frames 80 and 87) lay that choice open to the reader.

Source: The four combined misleading patterns are compiled from statistics and visualisation practice; they link to the causality theme of L12 in this course.

[Sample + Visualisation] Six Tricks, Six Fixes



Top row "misleading version", bottom row "fixed version": (1) truncated axis (2) bubbles by radius (3) false correlation from dual axes (4) only the favourable range (5) a pie chart for similar shares (6) uncertainty thrown away – one set of A/B data, six ways of telling the story.

Program `14_09_misleading.py` · sample `14_ab_test.csv` ($n = 2400$, group A 11.14% / group B 13.11%)

[Code] Chart Conventions as Assertions

```
checks = [                                  # every check returns a boolean
    ("y axis starts at zero", True),        # the fixed version satisfies this
    ("bubbles encoded by area, not radius", True), # area is proportional
    ("no dual axis for two quantities", True), # use a scatter to see the relation
    ("no selective range", True),           # plot every day
    ("no pie chart for similar shares", True), # use a bar chart
    ("uncertainty (confidence interval) shown", True), # with error bars
    ("sample size and absolute difference reported", True), # report n and the gap in points
]                                           # end of the checklist
assert all(ok for _, ok in checks), "chart convention check failed" # stop unless all pass
se_a = np.sqrt(rate_a * (1 - rate_a) / n_a)  # standard error of a proportion
se_b = np.sqrt(rate_b * (1 - rate_b) / n_b)  # standard error of the other group
CI_A = (rate_a - 1.96 * se_a, rate_a + 1.96 * se_a) # 95% confidence interval for A
CI_B = (rate_b - 1.96 * se_b, rate_b + 1.96 * se_b) # 95% confidence interval for B
overlap = not (CI_A[1] < CI_B[0] or CI_B[1] < CI_A[0]) # do the two intervals overlap?
print("do the intervals overlap:", overlap)  # if so, say "a hint, not a confirmation"
```

Measured on this lecture's sample: A's 95% CI = [9.39%, 12.89%], B's = [11.16%, 15.06%], the intervals overlap and the difference interval crosses 0 – so the correct wording is "there are signs of a lift, but it is not yet confirmed". Full program `14_09_misleading.py`.

Showing Uncertainty

Four common ways

- Error bars: point estimate $\pm$ standard error / confidence interval (on both bars and scatters);
- Confidence bands: an interval that moves over time (time series);
- Fan forecast bands: wider the further out you go (forecasting);
- Distribution comparison: draw the distributions directly (box / violin / density) – the most informative of all.

Three disciplines

1. state what the interval is (95% CI? standard error? quantile interval? bootstrap?);
2. state the number of resamples B and the method;
3. for a between-group comparison look at the difference interval, not just whether two intervals overlap.

"Not drawing uncertainty" is often more dangerous than "drawing it wrongly": readers take the point estimate as fact.

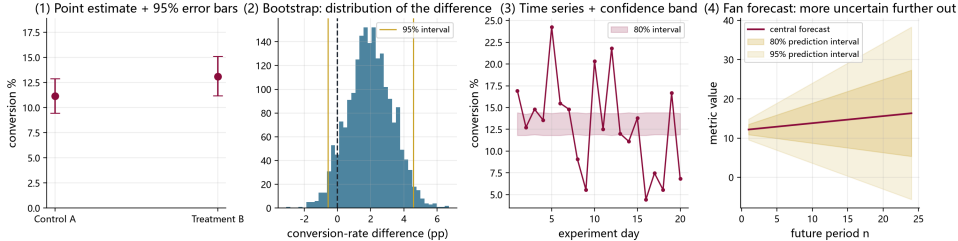
Source: The taxonomy of uncertainty visualisation follows statistical-graphics practice; the disciplines are added for this lecture (sample page 83).

[Code] Bootstrap Confidence Bands

```python
def bootstrap_ci(values, stat=np.mean, level=0.95, boot=B): # generic bootstrap interval
    values = np.asarray(values, dtype=float)          # cast to a float array
    idx = RNG.integers(0, len(values), size=(boot, len(values))) # resample the same size each time
    stats_ = stat(values[idx], axis=1)                # the statistic of each resample
    lo, hi = np.percentile(stats_, [(1 - level) / 2 * 100, # lower percentile bound
                                    (1 + level) / 2 * 100]) # upper bound
    return float(stat(values)), float(lo), float(hi), stats_ # point estimate, interval, distribution
m, lo, hi, dist = bootstrap_ci(sub["converted"])      # bootstrap interval for the conversion rate
diff_dist = boot_store["Treatment B"][3] - boot_store["Control A"][3] # bootstrap distribution of the difference
d_lo, d_hi = np.percentile(diff_dist, [2.5, 97.5])   # 95% interval of the difference
axes[2].fill_between(daily.index, band_lo, band_hi,  # time-series confidence band
                     color="#8A0C3C", alpha=0.18)
axes[3].fill_between(months, center - 2 * width,     # fan forecast band (wider further out)
                     center + 2 * width, alpha=0.15)    # 95% prediction band (still wider)
```

The advantage of the bootstrap: no distributional assumption is needed, and it gives both the interval and the shape of the distribution; the difference interval $[-0.0056, +0.0456]$ crosses 0, which is exactly why uncertainty must be reported. Full program `14_10_uncertainty.py`.

[Sample + Visualisation] Four Ways to Show Uncertainty



(1) point estimate + 95% error bar ($B = 2000$ bootstrap resamples); (2) the difference distribution and its 95% interval (crossing 0); (3) a time-series confidence band; (4) a fan forecast band (uncertainty widening over time).

Program `14_10_uncertainty.py` · summary table `14_uncertainty_summary.csv` · sample `14_ab_test.csv`

Colour-blind Friendly and Accessible Design

Three kinds of colour-vision deficiency, and the response

- red–green (most common): do not use a red–green contrast to show gains and losses;
- blue–yellow: avoid pairing blue with yellow;
- total colour blindness: the lightness difference must be large enough (distinguishable in greyscale).

Palettes that work

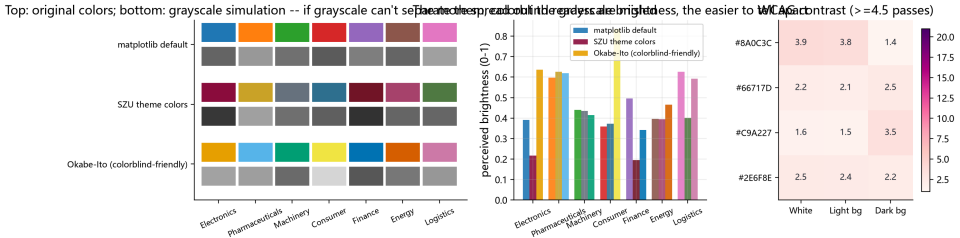
Okabe–Ito (8 colours, colour-blind friendly), viridis (continuous, monotone lightness), the colorblind-safe sets in ColorBrewer.

Six checks

1. add redundant encoding beyond colour (shape, line style, direct labels);
2. still distinguishable after greyscale printing;
3. text-to-background contrast ≥ 4.5 (large text ≥ 3.0);
4. font sizes on a projector: body $\geq$ 18pt, in-figure labels $\geq$ 12pt;
5. provide alt text for every chart (a one-sentence conclusion);
6. interactive charts offer keyboard-accessible filtering and zoom.

Source: Palettes and thresholds follow the Okabe & Ito palette and the WCAG 2.2 contrast requirements; the checklist is added for this lecture (sample page 85).

[Sample + Visualisation] Palette and Greyscale Check



Left: three palettes in colour and in greyscale simulation (colour on the top row, greyscale below); middle: the perceived lightness of each colour (the more spread out, the easier to tell apart); right: the WCAG contrast matrix of foreground against background ($\geq$ 4.5 passes).

Program `l4_11_accessibility.py` · report `l4_accessibility_report.csv` · checklist `l4_accessibility_checklist.csv`

Storytelling and AI-assisted Charting

The three-part structure of a data story

1. Context: the problem and the stakes (why the reader should care);
2. Conflict: what contrast the data reveals (this is where the chart enters);
3. Conclusion: the one sentence the reader should remember (the title is the conclusion).

How: write the conclusion in the title, annotate the key point, order the charts by reading order, and use scrollytelling where it helps.

AI-assisted charting (2026)

- text-to-chart: describe it in natural language → generate chart code or a chart;
- good for: quick exploration, first drafts, batch templating;
- not for: definitional judgements, the choice of statistical method, and the integrity check on the final version.

Verification checklist

Was the definition rewritten? Was the axis truncated? Is the sample size stated? Is it the data you supplied? Archive the prompt and the model version.

Source: The narrative structure follows the standard data-story framework; text-to-chart and the verification checklist are new for 2026 in this lecture and consistent with the course's AI-use policy.

Why High-dimensional Data Is Hard to Draw

Three difficulties

- Curse of dimensionality: the volume of space grows exponentially with the number of dimensions, so distances tend to "everyone is equidistant";
- Channels run out: a scatter plot has only two positional channels, so a third variable has to borrow colour or size;
- Overplotting: even pairwise, a 12-dimensional dataset gives $12 \times 11/2 = 66$ charts.

Three strategies

1. Dimensionality-reduction projection: compress 12 dimensions to 2–3 and draw (PCA / t-SNE / UMAP);
2. Multivariate profiles: parallel coordinates, radar charts, clustered heat maps, small multiples;
3. Hand it to interaction: 3D rotation, linked filtering, embedding-projection browsers.

This part runs through one 600 rows $\times$ 12 dimensions customer-behaviour dataset (3 latent groups, 48.8% churn): first how it "looks" (projection), then its profile (parallel coordinates), then why the model decides as it does (interpretable visualisation).

Source: The curse of dimensionality and the channel limits are basic results in visualisation and statistical learning; the sample and the conclusions in this part are measured in this lecture.

Dimensionality Reduction: PCA, t-SNE and UMAP

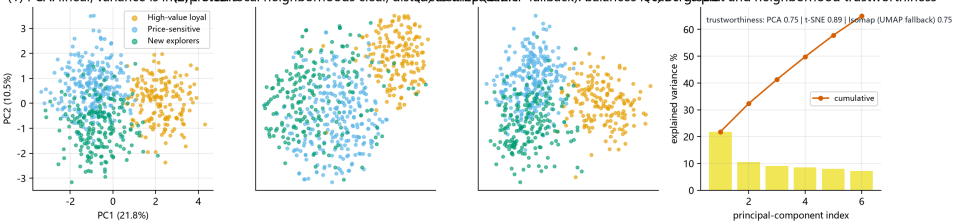
Method	What structure it keeps	Key points and cost
PCA	linear directions of maximum variance	interpretable (variance share), invertible, usable as features; only sees linear structure
t-SNE	local neighbourhoods (who resembles whom)	clusters separate well and look good; cluster distances and sizes are meaningless, high randomness, $O(n^2)$ and slow
UMAP	local + global structure	faster, can transform new data; sensitive to n_neighbors / min_dist, so fix the seed
Isomap / MDS	geodesic distance / distance fidelity	classic methods, a comparison and substitute for UMAP; sensitive to noise and the number of neighbours

Three general disciplines: (1) standardise before projecting (otherwise large-unit variables monopolise the variance); (2) write the parameters and the random seed into the caption; (3) judge a projection by neighbourhood fidelity (trustworthiness), not by "it looks well separated".

Projections distort. Always say how.

Source: Method positioning follows each method's official documentation (scikit-learn, the UMAP docs); the disciplines are compiled for this lecture (code page 90).

[Sample + Visualisation] Three Projections of One Dataset



The same 12-dimensional customer data: (1) PCA (the first two components explain 21.8% + 10.5%), (2) t-SNE, (3) UMAP, (4) the scree plot and neighbourhood fidelity (t-SNE 0.89 > UMAP 0.85 > PCA 0.75) – separating well is not the same as being faithful, so look at all three together.

Program `14_13_highdim.py` · sample `14_highdim_customers.csv` · metrics `14_highdim_projection_metrics.csv` · 3D interactive `interactive/embedding_3d.html`

[Code] Projections and Trustworthiness

```
X = customers.drop(columns=["segment", "churn"]).values # feature matrix (12 dimensions)
X_std = StandardScaler().fit_transform(X)             # standardise: required before projecting
pca = PCA(n_components=6, random_state=42).fit(X_std) # keep six principal components
Z_pca = pca.transform(X_std)[:, :3]                   # first three components (shared by 2D and 3D)
evr = pca.explained_variance_ratio_ * 100              # variance explained per component (%)
Z_tsne = TSNE(n_components=2, perplexity=30, init="pca", # t-SNE: emphasises local neighbourhoods
            learning_rate="auto", random_state=42).fit_transform(X_std) # a fixed seed makes it reproducible
Z_umap = UMAP(n_neighbors=15, min_dist=0.1, n_components=2, # UMAP: neighbours and min distance are key
            random_state=42).fit_transform(X_std)      # balances local and global structure
tw_tsne = trustworthiness(X_std, Z_tsne, n_neighbors=10) # fidelity: are near neighbours still near?
metrics = pd.DataFrame({"method": ["PCA", "t-SNE", umap_label], # summary table of projection quality
    "trustworthiness (10-NN)": [round(tw_pca, 3), round(tw_tsne, 3), round(tw_umap, 3)]}) # the three scores
dump(metrics, "14_highdim_projection_metrics.csv")     # write the metrics for the report and caption
```

Key point: a fixed four-step flow – standardise → project → score with a metric → write to disk; if UMAP is not installed the program falls back to Isomap without breaking. Full program `14_13_highdim.py`.

Reading a Projection, and Its Pitfalls

Five disciplines

1. cluster-to-cluster distances cannot be read: near and far in t-SNE/UMAP have no numeric meaning;
2. cluster sizes and densities cannot be read either (t-SNE artificially balances cluster sizes);
3. empty space means little: an area with no points is not necessarily an area that "should have none";
4. write the parameters and the seed into the caption: change perplexity, n_neighbors or the seed and the picture changes;
5. always cross-validate against business variables: do the "clusters" in the projection correspond to real groups?

Quality metrics

- Trustworthiness: how many of the 10 nearest neighbours in high dimensions are still near in low dimensions;
- Continuity: the reverse check (are low-dimensional neighbours also near?);
- Residual variance (PCA only): what percentage the first k components explain;
- when reporting, give the metric and the chart together, never the chart alone.

In one sentence: a projection is a tool for seeing structure, not evidence for a conclusion – conclusions must be verified on statistics in the original dimensions.

Source: Common misreadings of projections and the quality metrics (trustworthiness / continuity) are compiled from manifold-learning and visualisation-evaluation research.

Parallel Coordinates and Radar Charts

Parallel coordinates (recommended)

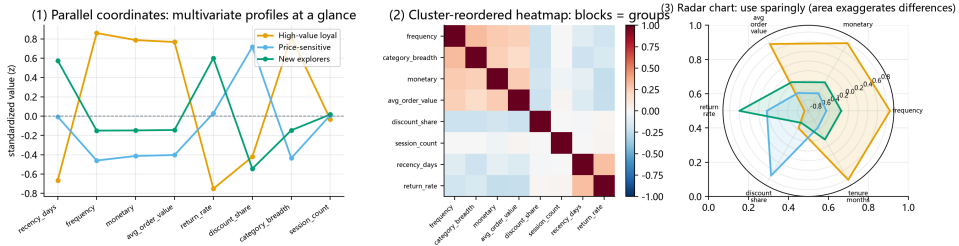
- one vertical axis per variable, one polyline per observation: see 8–12 variables at once;
- they must be standardised (otherwise the axes have different units and the shape is driven by the units);
- axis order affects the impression: crossings between adjacent axes are easiest to read, so order by correlation;
- when grouping categorically, draw group mean lines plus a few individual lines instead of smearing a thousand lines together.

Radar charts (use with care)

- essentially parallel coordinates bent into a circle: the area changes with axis order and scaling, so areas are not comparable;
- with many axes the shapes are all alike and the reader can only see "which spike is longest";
- if you must use one: fix the axis order, unify the units, keep axes ≤ 6, and give the numeric table as well.

Source: The reading limits of parallel coordinates and radar charts are compiled from visualisation practice and statistical-graphics criticism; the sample is on the next page.

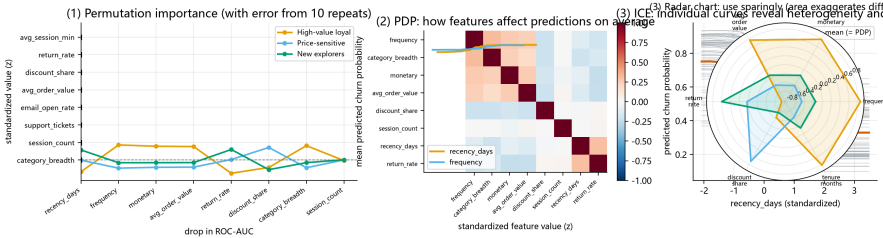
[Sample + Visualisation] Multivariate Profiles: Parallel Coordinates, Clustered Heat Map, Radar



(1) Parallel coordinates (8 standardised features, the mean profile of three groups): the loyal group is higher on frequency, amount and tenure throughout; (2) the cluster-reordered correlation heat map (variables that form a block belong together); (3) a radar chart – note that the area is exaggerated, so use it only illustratively.

Program `14_13_highdim.py` · sample `14_highdim_customers.csv` · hierarchical reordering uses `scipy.cluster.hierarchy`

[Sample + Visualisation] Model Interpretation: Permutation Importance, PDP and ICE



A random forest predicting churn (n=600, churn rate 48.8%): (1) permutation importance (error bars over 10 repeats) – days since the last purchase at 0.195 far exceeds the runner-up at 0.075; (2) PDP gives the average marginal effect; (3) ICE shows individual differences: an average fall does not mean everyone falls.

Program `14_13_highdim.py` · importance table `14_highdim_importance.csv` · model and metric: `permutation_importance` from `scikit-learn`

[Code] Permutation Importance, PDP and ICE

```
clf = RandomForestClassifier(n_estimators=400, min_samples_leaf=4, # random forest as an interpretable model
                             random_state=42).fit(X_std, customers["churn"]) # trained on the churn label
perm = permutation_importance(clf, X_std, customers["churn"], # permutation importance: shuffle a feature
                              n_repeats=10, random_state=42, scoring="roc_auc") # and see how much performance drops
imp = pd.DataFrame({"feature": list(FEATURES.keys()), # feature names
                    "importance": perm.importances_mean.round(4), # mean importance
                    "std": perm.importances_std.round(4)}) # spread across the repeats
for v in qs:                                          # loop over the quantile grid
    X_tmp = X_std.copy()                              # copy the feature matrix
    X_tmp[:, j] = v                                   # replace that whole column with v
    preds.append(clf.predict_proba(X_tmp)[:, 1].mean()) # mean predicted probability = PDP
for r, i in enumerate(idx_sample):                    # draw ICE for individuals
    for k, v in enumerate(qs):                        # over quantiles
        X_tmp = X_std[i:i + 1].copy()                 # take just that individual's row
        X_tmp[0, j] = v                               # change the feature value
        ice[r, k] = clf.predict_proba(X_tmp)[0, 1]    # predicted probability for that individual
fig3d.write_html(INT / "embedding_3d.html", include_plotlyjs="cdn") # 3D embedding: rotate to explore
```

How the three charts divide the work: importance answers "which variable matters", PDP answers "how it affects things on average", and ICE answers "is it the same for everyone". Full program `14_13_highdim.py`.

Frontier and Checklist

High-dimensional visualisation in 2026

- Embedding-projection browsers: reduce text / image / user vectors to 2–3 dimensions and search interactively (sentence vectors, multimodal embeddings such as CLIP, item vectors in recommender systems);
- 3D and animation: rotatable 3D embeddings (this lecture's `embedding_3d.html`), grand-tour animations that scan high-dimensional space;
- interpretable visualisation is now standard: SHAP summary plots, PDP/ICE, attention heat maps, counterfactual explanations (linking to L9 model evaluation and L13 interpretability);
- accessibility extends to high-dimensional charts: shape and labels beyond colour, keyboard access and text summaries for interactive charts.

High-dimensional visualisation checklist

1. did you standardise before drawing?
2. are the projection's parameters and seed in the caption?
3. did you give a quality metric (trustworthiness / variance explained)?
4. did you treat "cluster-to-cluster distance" as a conclusion?
5. did you cross-validate the grouping against business variables?
6. is the interpretation chart model-agnostic or built in? Is that stated?
7. does the caption say that "correlation is not causation"?

Summary: expressing high-dimensional data relies on three complementary means – dimensionality-reduction projections (seeing structure), multivariate profiles (seeing individuals and groups), and interpretability visualisation (seeing the model's basis); any of them needs metric + caption + cross-validation to be trustworthy.

Source: The frontier directions are compiled from visualisation and interpretable-machine-learning practice; the checklist is compiled for this lecture.

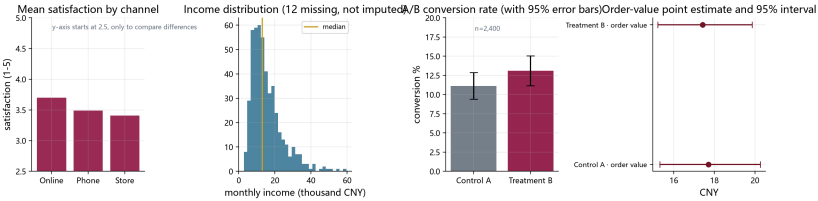
Ten Checks for an Honest Chart

1. does the chart type match the question (comparison/distribution/relationship/time/space/structure)?
2. does the vertical axis start at 0 (bar charts), or is the truncation declared (line charts)?
3. does each value use one channel only (no double encoding)?
4. have you used dual axes to manufacture a false correlation?
5. is the range complete (no selective truncation)?
6. is the uncertainty drawn (interval/error bars) and the method stated?
7. are the sample size, the missing-value treatment and the definitions in the caption?
8. is the colour colour-blind friendly, greyscale-distinguishable and redundantly encoded?
9. are the title and caption "self-contained" (readable without the body text)?
10. is the data reproducible (scripts, seeds, versions)?

How to use it: treat this table as your last-minute self-check before submitting a report – tick all ten before you hand it in.

Source: The ten checks bring together all of Part 6 and can serve as a self-check table for project reports.

Project 1: Charts and Description



A four-panel main figure for the report: satisfaction by channel (with the truncated axis declared), the income distribution (noting that missing values were not imputed), A/B conversion rates (with 95% error bars and n), and the average-basket interval – the accompanying statistics table and captions are exported by the same program.

Program `14_12_report_pipeline.py` · report material `data/lecture04/report/` (table1, table2, captions.md)

Common Chart Mistakes: Top 10

1. a bar chart whose vertical axis does not start at 0;
2. a dozen categories and a dozen colours crammed into one chart;
3. using a pie chart to compare similar shares;
4. dual-axis line charts that manufacture a false correlation;
5. plotting only the favourable range or reporting only the significant subsample;
6. no sample size and no uncertainty;
7. a caption that is not self-contained (the reader cannot find the units or the definitions);
8. a rainbow palette for an ordinal variable;
9. a red–green contrast with no redundant encoding (colour-blind readers cannot tell);
10. a conclusion that does not match the chart (the chart shows no "significant improvement").

These ten map one-to-one onto the checklist on frame 97: read the mistake list first, then accept the work with the checklist.

Source: The mistake list is compiled from the controlled experiments in Parts 4–6 of this lecture and from common reviewer comments.

Toolbox: Quick Reference

Task	Key API / tool
Descriptive statistics	`describe()`, `quantile()`, `skew()`, `kurt()`, `groupby().agg()`
Univariate charts	`hist`, `boxplot`, `violinplot`, `step` (ECDF)
Relationship charts	`scatter`, `corr()` + `imshow` (correlation matrix), `hexbin`
Time charts	`plot`, `rolling().mean()`, `fill_between` (confidence bands)
Maps	`px.choropleth` (ISO-3), `go.Scattergeo` (symbol/flow), GeoJSON polygons
Interaction	`px.line/scatter`, `go.Sankey`, `make_subplots`, `write_html`
Accessibility	the Okabe–Ito palette, greyscale conversion `0.299R+0.587G+0.114B`, the WCAG contrast formula

All examples live in `website/code/lecture04/`: 12 programs, 15 figures and 6 interactive HTML files, commented line by line and rerunnable in one command.

Source: The API quick reference follows the matplotlib / seaborn / plotly documentation; the examples correspond to the code pages of this lecture.

Glossary and References

中文	English
集中趋势	Central tendency
离散程度	Dispersion
偏度 / 峰度	Skewness / Kurtosis
列联表	Contingency table
受限范围	Restricted range
视觉通道	Visual channel
过绘	Overplotting
仪表板	Dashboard
分级统计图	Choropleth
空间自相关	Spatial autocorrelation
扇形预测带	Fan chart
替代文本	Alt text

References

- Rahlf, T. *Data Visualisation with R* (set reading for L4 in the syllabus)
- McKinney, *Python for Data Analysis*, Ch. 9 Plotting and Visualization
- Cairo, A. *How Charts Lie*; Tufte, *The Visual Display of Quantitative Information*
- Wilke, C. *Fundamentals of Data Visualization*; Munzner, *Visualization Analysis and Design*
- Public resources: data-to-viz, the python/r graph galleries, WITS, Gephi, CiteSpace, SNAP, GeoDa, QGIS
- WCAG 2.2 (contrast and non-text content); the Okabe & Ito colour-blind friendly palette

Source: The references and resources match the L4 reading requirements in the syllabus; the external links are on frames 74 and 62.

Thank you!

Q & A · Data Description and Visualization

This lecture in one line: documentation and metadata → centre, dispersion, shape → correlation and multivariate → charts and channels → interaction, dashboards, maps → honest, accessible, well told

Class checkpoint: show only the chart, not the body text, and ask a classmate to restate your conclusion; if they cannot, go back and fix the chart.

Samples and programs: `website/code/lecture04` (12 programs, commented line by line) `website/data/lecture04` (28 sample files) `website/figures/lecture04` (15 figures + 6 interactive HTML files)

Lecturer: Xieer Dai (David) · xieer.dai@szu.edu.cn · Sifang Building South 207